

SOTA 分析：長 context 的 KV Cache 分層（附檔）

一句話：L2「KV 放在哪一層」的招式已經很多，但幾乎都在讀取或逐出時決定、評測在 16K–128K、I/O 用「大小 ÷ 頻寬」；寫入時、依位置、多層、長 context 的組合還空著。

日期：2026-10-06 主文：[RESEARCH_INTRO_20261006.md](#)。本檔放主文放不下的細節。標註：[n] 原文，編號對應文末的參考文獻（IEEE 格式）；〔計算〕我們從公開資料或論文數字算出；〔推論〕我的判讀。所有數字都是各論文在自己的硬體上量的，不能跨列比較。讀到多深：〔全文〕讀過全文；〔部分〕讀過相關段落或多個查證來源交叉確認；〔摘要〕只讀摘要，細節引用前要回原文確認。

1. 範圍

- 主要：跨請求重用的 KV 管理，也就是 L2「放在哪一層」，以及和它直接相關的 L3（傳輸）、L4（命中規則）。
- 對照：會被拿來比較的 L0（物件）、L1（精度）代表作，以及「單一請求內丟 token」的學習式方法。
- 不含：本專案的硬體量測（另有報告）。

2. L2 核心論文：逐篇卡片

每篇分「在解什麼 / 怎麼做 / 強在哪 / 做不到 / 跟本研究的關係」。

2.1 Cake (ICML'25) [1] 〔全文〕

- 在解什麼：長 context 的 KV 存在儲存裝置上，要用時只能全部載回（受頻寬限制）或全部重算（受算力限制）。
- 怎麼做：GPU 從前面往後重算，I/O 從後面往前載入，兩邊同時跑，在中間會合。依據是：越後面的 token 重算越貴，載入成本跟位置無關。會合點在執行時自動決定，GPU 被別人佔用或頻寬變動時會自己移動。
- 強在哪：
 - TTFT 平均比只算或只載快 2.6 倍；
 - 不用訓練、不用事先量；
 - 掃了 I/O 頻寬（7–100 Gbps）、GPU 使用率（12.5%–100%）、長度（4K–16K）、模型（MHA / GQA）、量化（8-bit / 3-bit）。
- 做不到：
 - I/O 用延遲模擬，沒有接真實裝置；
 - 假設所有請求的 KV 都事先存好；
 - 只有一個儲存層；
 - 長度只到 16K（合成 prompt，原文認為只有長度會影響結果）；

- 沒有退路：Llama-3.1-70B、GPU 只剩 12.5%、32 Gbps 時比只載還慢（0.80 倍），作者把「估計機制 + 退回單一路徑」列為未來工作；
- 沒有公開程式碼。
- 跟本研究的關係：起點。我們把它讀取時的規則提前到寫入時，並延伸到多層與長 context。

2.2 CacheFlow (arXiv 2604.25080 v2, 2026-09) [2] [全文]

- 在解什麼：KV 還原在多輪對話、agent、RAG 中成為 TTFT 的主要瓶頸；現有做法把還原當成「整個請求要算還是要載」的粗粒度選擇。
- 怎麼做：
 - 把還原看成 token、層、GPU 三個維度上的平行執行；token 維度就是 Cake 的做法，層維度類似 HCache；兩者合起來是「階梯」形的切分；
 - 用批次感知的動態規劃，一次規劃整批請求的所有（請求, 層區塊），讓它們共用 GPU 與 I/O；
 - 重算成本從最近的服務紀錄線上量測。
- 強在哪：
 - TTFT 平均快 2.24–3.00 倍；比 Cake 快 2.40–2.63 倍（40 / 80 Gbps）；
 - 預測準：92% 的重算預測、98% 的載入預測誤差在 10% 內；95% 的還原，重算和載入在 10% 內同時完成；
 - 有理論：和理想最佳解的差距是 $O(1/N)$ ；
 - 處理了 hybrid 模型（Qwen3.5 的 recurrent 層）。
- 做不到：
 - 不處理寫入：假設所有狀態都可以載入，被逐出的只能重算；
 - 只有一個載入頻寬，傳輸時間是位元組 ÷ 頻寬；
 - 理論假設計算和載入兩種資源互不干擾，只處理同一批還原請求之間的競爭；
 - 所有設定都報加速，沒有失效的區域；
 - 程式碼要等論文被接受後才釋出。
- 跟本研究的關係：讀取端的最強對手。我們不再做讀取規劃器，直接把它當上界與基準；我們的空間在寫入端。注意：它的符號 κ 和我們的「重算 ÷ 傳輸成本比」意思不同。

2.3 Strata (OSDI'26) / SGLang HiCache [3] [全文]

- 在解什麼：長 context 時，KV 已經在 CPU 記憶體裡，但系統卡在搬回 GPU：KV 被切成很多小 page，搬運變成大量小傳輸；載入會卡住 prefill；同一段 KV 正在載入時又被請求（delayed hit），排程器卻不知道。
- 怎麼做：
 - GPU 協助搬運，GPU 端和主機端用不同的資料版面，讓小塊能整批傳；
 - 排程器同時看「要載入的」和「要計算的」，把搬運和計算錯開；
 - 寫入有三種策略：只在要被逐出時才備份、每次都備份、被存取超過門檻（預設 2）次才備份（預設）；
 - 每一層的逐出都用 LRU。

- 強在哪：吞吐量比 vLLM + LMCache 最多高 5 倍；已在 SGLang 的生產環境使用；用 Little's Law 說明「每次搬運的量要放大」。
- 做不到：放哪、丟誰只用 LRU；不把重算當選項，原文認為長 context 的重算不划算；沒有量化；SSD 只出現在一個子實驗。
- 跟本研究的關係：最強的搬運機制；也是「寫入時依存取次數決定」的生產做法，我們改成依位置與預測決定。

2.4 Pensieve (EuroSys'25) [4] [部分]

- 在解什麼：多輪聊天時，GPU 放不下所有對話的歷史 KV，要決定先丟誰。
- 怎麼做：優先丟「很久沒說話」和「重算比較便宜」的對話，而且從對話的開頭丟起；丟掉的前段要用時再重算。
- 強在哪：吞吐量是 vLLM、TensorRT-LLM 的 1.14–3.0 倍；最早把「位置不同，重算成本就不同」用在逐出上。
- 做不到：只有 GPU 和 CPU 兩層，沒有 SSD，也沒有精度；沒有開源。
- 跟本研究的關係：「從前段逐出」是 E1 必須分離的對照組（P2）；如果它就拿走大部分好處，本研究的新穎性會變窄。

2.5 Bidaw (FAST'26) [5] [全文]

- 在解什麼：聊天服務裡，使用者回覆前會停一段時間，這段期間 KV 被放到 SSD；下一輪要從 SSD 拿回，比「全部放記憶體」慢最多 3.8 倍。
- 怎麼做：
 - 計算端與儲存端互相知道對方的狀態，依 KV 在記憶體還是 SSD 分開排程；
 - 用「上一輪回答多長」預測使用者多久後回來：回答越長，讀得越久，KV 越早移到 SSD；
 - 背景跑一個 Belady 影子快取，線上校準策略；
 - 改存可以換算回 KV 的中間 tensor（只對 MHA 划算）。
- 強在哪：延遲最多降 3.58 倍；在超過一百萬輪的真實聊天 trace 上，用 Belady 證明「聰明的放置確實有空間」；trace 公開。
- 做不到：「回答越長、越晚回來」只對真人聊天成立，原文承認換成 Poisson 到達就失效；問題平均只有 36 token；改存中間 tensor 對 GQA 模型不划算。
- 跟本研究的關係：「預測何時回來」的先例。agent 負載需要不同的訊號（工具類型、等待時間）。

2.6 MTDS (Complex & Intelligent Systems, 2026) [6] [全文]

- 在解什麼：邊緣伺服器只有 24 GB 的小卡，頻寬也小，從 SSD 載入有時候比重算還慢。
- 怎麼做：用歷史延遲預測每個請求的載入時間和重算時間，再選全載、只載前面一部分，或乾脆不載；被越多排隊請求共用的 KV 越優先卸載。
- 強在哪：TTFT 降超過 25%。
- 做不到：整段一起決定，不依位置切開；模型最大 14B；用 ShareGPT 加 Poisson 到達。
- 跟本研究的關係：它的結論（命中很長時不載反而快）和 Strata 相反，很可能來自各自的傳輸速度〔推論〕。

2.7 AdaptCache (BigMem'25) / EvicPress (arXiv 2512.14946) [7], [8] [全文]

- 在解什麼：KV 太多，只能一部分放 DRAM、其他放 SSD；要不要壓縮、壓多少、放哪裡。
- 怎麼做：效用 = 預估的未來命中頻率 × (品質的權重 - 大小 ÷ 頻寬)，用 greedy 求解；對壓縮敏感的少壓、放上層。
- 強在哪：最早主張「壓縮和放置一起決定」；EvicPress 同品質下 TTFT 最多快 2.19 倍。
- 做不到：未來命中頻率用過去的命中頻率估；品質評估很薄 (AdaptCache 每個資料集抽 10 筆)；兩篇都沒算上界；EvicPress 承認 DRAM 夠大或各層速度相近時優勢消失；兩篇的資料設定都無法照原文重現 [計算，見 §4.5]。
- 跟本研究的關係：唯一把精度當成放置決策的一部分；我們改成依位置決定精度。

2.8 LMCache [9]、Mooncake (FAST'25) [10] [全文]

- **LMCache**：業界最常用的開源 KV 快取層，被 vLLM、SGLang 整合；搭配 vLLM 吞吐最多高 15 倍。策略不是它的重點；大倍數出現在對手已經接近飽和的工作點。
- **Mooncake**：Kimi 的生產架構，prefill 和 decode 分開，把叢集閒置的 CPU 記憶體和 SSD 池化成 KV 倉庫；讓 Kimi 多處理 75% 的請求；公開真實 trace (每個 hash block 是 512 token)。需要多節點和 RDMA。

2.9 py-kvcache (arXiv 2609.11744, 2026-09) [11] [摘要+部分]

- 在解什麼：外部 KV 快取 (NVMe SSD) 什麼時候值得用？
- 怎麼做：在 vLLM + NVMe 上量出「存 SSD 比重算划算」的損益平衡長度；短於這個長度的前綴就不存也不載。
- 強在哪：明確主張外部 KV 快取應該被當成依每套硬體設定而定的准入決策；在較弱的 GPU 上改善 TTFT，在 H100 上平均請求低於損益平衡點、GPU 記憶體就夠用。
- 做不到：以整段前綴長度判斷，不看 block 位置；沒有精度。
- 跟本研究的關係：「先量常數、再決定」的想法最接近，必須引用並劃清界線 (我們依位置、多層)。

2.10 AsymCache (arXiv 2606.02964) [12] [摘要]

- 怎麼做：逐出時同時考慮命中率和「依位置而變的重算成本」；用 Multi-Segment Attention 支援不連續的 KV。
- 強在哪：TTFT 最多快 1.90–2.03 倍，TPOT 快 1.62–1.71 倍；整合進 agent 服務系統時，平均工作延遲最多降 18.1%。
- 跟本研究的關係：「前段不存 → 需要區段命中」的最接近前作；也是位置感知逐出的對手。要讀全文。

2.11 When Fancy Eviction Fails (arXiv 2609.28870, 2026-09) [13] [摘要+部分]

- 在解什麼：在真實的生產 trace 上，各種精巧的逐出策略到底有沒有比 LRU 好？
- 資料：兩份生產 trace (FreeInference：7 天、32.75 萬個請求、平均 32.0K token，以 agent 為主；Chutes：130.9 天、51.58 萬個請求、平均 18.7K token)。
- 發現：
 - 14 種逐出法 (含學習式的 LeCaR、LRB、3LCache) 大多沒有比 LRU 好，看頻率的常常更差；
 - 因為前綴重用主要由 session 的節奏決定，「最近用過」本身就很會預測；

- Belady 上界在各種容量下仍有明顯空間；
- 建議以 LRU 為底，加三個針對性改進：只用一次的前綴快速降級、成本高的未命中做成本感知的部分逐出（從節點開頭整段丟，比 LRU 少 19.9% TTFT）、依容量調整粒度；
- session 壽命短（中位數不到 1 分鐘，p99 約 4 小時），所以對目前多數負載，SSD 卸載沒有必要。
- 跟本研究的關係：兩面都有：
 - 支持「OS 派的 LRU 是強基準」「學習式要證明自己」；
 - 「從開頭整段逐出」是位置感知的近親；
 - 「SSD 沒必要」是本研究前提的反例。我們的前提是「長 context + 長間隔」，必須用 agent trace 檢驗它在真實負載中佔多少。

2.12 HCache (EuroSys'25) [14]、KVPR (ACL Findings'25) [15] [全文／部分]

- **HCache**：不存 KV，改存每層的 hidden state（MHA 下約 KV 的一半），要用時做一次矩陣乘法轉回 KV；TTFT 比 KV 卸載快最多 1.93 倍；只適用 MHA。它在「層」的維度實作了「會被重算的就不存」。
- **KVPR**：KV 在 CPU 時，先傳一部分 activation 讓 GPU 開始重算，其餘 KV 同時傳輸；profiler 依輸入長度和硬體算出最佳切點。只支援單 GPU、不處理從磁碟載入、只在一開始量一次並假設硬體不變。

3. 預測派細節

3.1 誰預測什麼、要付多少訓練成本

方法	類型	預測什麼	訓練／更新	保底	決策的層次
LRB (NSDI'20) [16]	GBDT	多久後再被存取（取 log 回歸）	CPU；每累積 128K 筆標記樣本就訓練新模型取代舊的；原文說負載會變，所以必須定期重訓	無	CDN 物件逐出
LCR / LARU (arXiv 2509.20979) [17]	GBDT (LightGBM)	逐出優先順序	CPU 上線上訓練	有：偵測到預測錯就退回 LRU；1-consistent、O(k)-robust	SGLang 的 KV 前綴樹逐出
SAECache (arXiv 2605.18825) [18]	線上學習	依 token 類型與位置給每個 block 保留分數	不需要外部嵌入模型、不需要定期重訓，參數線上更新	—	前綴快取逐出
Marconi (MLSys'25) [19]	啟發式	依命中情境估重用機率	線上調整權重	有：剛啟動時先用 LRU	hybrid 模型的准入與逐出
Bidaw (FAST'26) [5]	啟發式	使用者多久後回來（用上一輪回答長度）	背景的 Belady 影子快取線上校準	部分	DRAM / SSD 放置

方法	類型	預測什麼	訓練／更新	保底	決策的層次
MTDS (2026) [6]	回歸	載入時間與重算時間	用歷史延遲	—	全載 / 部分載 / 不載
CacheFlow (2026) [2]	量測	重算與載入的時間	從最近的服務紀錄線上量測	—	讀取時的切分
LPC (NeurIPS'25) [20]	深度學習	對話會不會繼續	訓練一個 118M 參數的文字嵌入模型 (SAECache 的描述 [18])	未查證	前綴快取逐出 (對話層級打分)
KVP (ICML'26) [21]	深度學習 (RL)	每個 token 的保留順序	每個 (層, KV head) 一個約 65 萬參數的 agent ; 收 trace 用 7 台 $\times$ 8 GPU (不到 2 小時、約 1.2 TB) , 訓練用 8 張 H100 (不到 30 分鐘)	—	單一請求內丟 token
ForesightKV (ICML'26) [22]	深度學習 (監督 + RL)	KV 的長期貢獻	先監督學習、再 GRPO 強化學習	—	單一請求內丟 token
LookaheadKV (ICLR'26) [23]	深度學習 (LoRA)	token 的重要度	訓練 lookahead token 與 LoRA	—	單一請求內丟 token
TRIM-KV (arXiv 2512.03324) [24]	深度學習 (gate)	token 的保留分數	凍結 LLM、只訓練 retention gate ; 4 張 H100 , 最長訓練到 128K	—	單一請求內丟 token
HALP (NSDI'23) [25]	學習 + 啟發式	逐出偏好	—	有 : 先用 LRU 篩候選	CDN 物件逐出

表 S1. L2 預測派與相關方法的比較：預測目標、訓練與更新方式、有沒有保底，以及決策的層次。

3.2 對「預測派都要大量訓練，換 server 就要重訓」的求證

說法	判斷	理由
深度學習派訓練很重	成立	KVP、TRIM-KV 都要多張資料中心級 GPU [21], [24]
深度學習派換模型要重訓	成立	它們讀的是特定 LLM 的 K / V / attention；KVP 對 Qwen2.5-7B 要 112 個 agent [21]
GBDT 派也要大量訓練	不成立	在 CPU 上訓練；LRB 每 128K 筆樣本換一次模型 [16]；LARU 線上訓練 [17]
換 server 就要重訓	看預測什麼	預測負載的模型與硬體無關；預測時間的部分才跟硬體綁在一起（KVPR 只量一次並假設硬體不變 [15]、MTDS 用歷史延遲 [6]、CacheFlow 線上量測 [2]）
最大的問題是重訓	不是	更大的問題是「預測準 ≠ 決策好」：多數系統盲從預測，錯的時候比 LRU 差 [17]；生產 trace 上學習式逐出沒有比 LRU 好 [13]

表 S2. 對「預測派需要大量訓練、換一台 server 就要重訓」的查證（同主文表 10）。

設計原則：硬體成本用量的、模型只學負載；成本改變時只移動決策門檻（成本敏感學習的經典結果 [26]），預測沒把握時退回規則（學習增強 [27], [28]）。

3.3 一個關鍵的資料觀察

長 context 的公開 agent trace，重用率都在 94–99.5%〔計算〕（§4.4）。如果真實的長 context 本來就以高重用為主，「會不會再用」就不是重點；真正要預測的是何時回來、回來時 GPU 多忙，以及誰該留在快的層〔推論〕。

4. 評測設定盤點（35 篇）

整理自 research_20260924/workloads_eval.md（有頁碼與計算方法）。

4.1 baseline 幾乎不重疊

- 有 baseline 的 32 篇共用了 64 種不同的 baseline，其中 41 種只出現在一篇。
- 496 個論文配對中，72.6% 的 baseline 集合完全不交集，平均 Jaccard 只有 0.056。只看多層放置 / 跨請求重用的 20 篇，零交集仍有 57.4%。
- 最常見的共同對手只有 LRU（9 篇）和 vLLM（9 篇）。ARC：0 篇。
- Strata、Bidaw、MTDS、AdaptCache、EvicPress、Cake、KVPR、HCache、Pensieve：35 篇中沒有任何一篇拿它們當 baseline。

4.2 請求到達

類別	篇數
主實驗用真實時間戳	2 (Bidaw、Mooncake)
只在子實驗用真實時間	2
真實 trace 但只取長度、時間被壓縮	1
Poisson	10
固定或手調間隔	4
分布未說明	3
沒有到達過程（單請求或批次）	19

表 S3. 35 篇論文主實驗的請求到達方式。

4.3 命中率沒有共同定義

- 15 篇報了命中率，粒度至少 6 種：token 加權前綴、區塊、請求級、單層或任一層、decode 時 top-K 已在 GPU 的比例、相鄰步驟重疊。
- 同一份 Mooncake trace（無限容量），依定義可以是 1.1%（請求完整命中）、55.3%（區塊）、57.1%（token）、66.9%（逐請求平均）或 100%（請求至少命中一塊）〔計算〕。

4.4 公開 trace

trace	時間	內容／單位	長度	重用
Mooncake (FAST'25) [10]	有	前綴 block hash， 512 token 一塊	conversation 中位 6,909、最大 126,195；toolagent 中位 6,346 〔計算〕	中（37–57%）〔計 算〕
Alibaba Bailian (ATC'25) [29]	有	16-token 的 SipHash block； 有對話 id	四條 trace 的輸入中位 574– 4,540；≥32K 的只有 0–0.18% 〔計算〕	中（46–67%）〔計 算〕
Bidaw [5]	有	只有長度	含歷史的 prompt 中位 1,066、 p99 8,168 〔計算〕	高（假設 prompt = 完整歷史）
TraceLab [30]	有，逐事件	沒有內容；有前綴 / 新增 的切分	輸入中位 124,018、p90 256,767；60.9% ≥100K 〔計 算〕	高（95.7%）
Codex SWE-bench Pro	只有統計	有內容	每次呼叫輸入中位 63,917	高（94.2%）
MLPerf Agentic	有延遲欄位	有內容（生成或合成）	DeepSWE 中位約 84K（估）	高（99% 以上）

表 S4. 公開 trace 的時間、單位、長度與重用率。〔計算〕為我們從公開資料算出；重用率為無限容量下的估計。

缺口：長 context（中位 ≥40K）+ 真實時間 + 中低重用的公開 trace，目前找不到。

4.5 資料設定無法重現的例子

- AdaptCache：原文沒有列出資料集名稱與筆數的對應，用官方資料湊不出它說的 1,100 段 context；程式碼與 query 都沒有公開〔計算〕。

- EvicPress：用 12 個 LongBench 資料集、555 段 context，每段由 GPT-5 生 100 題；Table 2 的長度沒有標單位，比對後應該是字元數，約只有 25K token 量級〔計算〕；程式碼與 query 沒有公開。

4.6 已有的統一評測嘗試

- 有人做了 L1（壓縮）的跨方法基準測試，同時看任務品質和系統效能 [31]。
- SCBench 量共享 context 下的品質，但重用率是構造出來的 [32]。
- 還沒有人同時標準化「CPU / SSD 分層 + 跨請求重用 + 品質」。

5. 新穎性威脅（依威脅程度）

威脅	它做了什麼	留給我們的
高：CacheFlow [2]	讀取時的三維還原、可預測的成本模型、批次規劃	寫入時的決定；多層；對讀取時 GPU 負載的穩健性
高：Fancy-eviction 研究 [13]	生產 trace 上 LRU 很強；從開頭整段逐出的成本感知法	寫入時；多層；長 context + 長間隔的負載
高：AsymCache [12]	位置感知的重算成本逐出；不連續 KV 的注意力	寫入時；多層；預測
中：Pensieve [4]	從對話開頭逐出	SSD 與精度；寫入時
中：py-kvcache [11]	依硬體量的損益平衡、整段准入	依位置；多層
中：LARU [17]	學習式逐出 + 有保證的退回	多層、成本隨位置變
中：Strata [3]	寫入時依存取次數決定要不要備份	依位置與預測決定
低：HCache [14]	在層的維度決定哪些不存	在位置的維度；不限 MHA
低：KVPR [15]	傳 activation 換重算起點	多層、長 context

表 S5. 新穎性威脅，依威脅程度排列。

結論：每個零件都有前作，空白只在組合。動手前要再做一次查新（上次是 10/4）。

6. 能不能重現：開源狀態

方法	程式碼	備註
Cake	沒有找到	作者頁只有 PDF；CacheFlow 是在 LMCache 上自己重做 Cake
CacheFlow	接受後才釋出	—
Pensieve、HCache	沒有找到	要自己重做
Strata / HiCache	SGLang 上游	—
LMCache、Mooncake、CacheBlend	有	—
Marconi、LPC、KVPR、py-kvcache、Tutti	有	LPC 改的是 vLLM 0.7.3
LRB	有（模擬器）	—
vLLM、SGLang、NVIDIA Dynamo	有	生產規則可直接跑

表 S6. 主要方法的程式碼公開狀態（2026-09-24 查證，見 research_20260924/novelty_sota.md）。

參考文獻（IEEE 格式）

[1] S. Jin, X. Liu, Q. Zhang, and Z. M. Mao, "Compute or load KV cache? Why not both?" in *Proc. 42nd Int. Conf. Mach. Learn. (ICML)*, 2025. [Online]. Available: <https://arxiv.org/abs/2410.03065>

[2] S. Nian, H. Shen, Z. Wu, J. Fang, Q. Feng, and F. Lai, "CacheFlow: Efficient LLM serving via automated 3D-parallel KV cache restoration," arXiv:2604.25080v2, Sep. 2026.

[3] Z. Xie *et al.*, "Strata: Hierarchical context caching for long context language model serving," in *Proc. 20th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2026.

[4] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025.

[5] S. Hu, G. Zhang, Y. Zhou, Y. Wei, Z. Zhong, and J. Chen, "Bidaw: Enhancing key-value caching for interactive LLM serving via bidirectional computation-storage awareness," in *Proc. 24th USENIX Conf. File Storage Technol. (FAST)*, 2026.

[6] J. Wang, J. Hu, Q. Cao, Y. Zhu, and X. Lin, "Multi-tier dynamic storage of KV cache for LLM inference under resource-constrained conditions," *Complex Intell. Syst.*, vol. 12, no. 3, Art. no. 104, 2026, doi: 10.1007/s40747-025-02200-4.

[7] S. Feng *et al.*, "AdaptCache: KV cache native storage hierarchy for low-delay and high-quality language model serving," in *Proc. SOSP Workshop Big Memory (BigMem)*, 2025.

[8] S. Feng *et al.*, "EvicPress: Joint KV-cache compression and eviction for efficient LLM serving," arXiv:2512.14946, 2025.

[9] Y. Liu *et al.*, "LMCache: An efficient KV cache layer for enterprise-scale LLM inference," arXiv:2510.09665, 2025.

[10] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot," in *Proc. 23rd USENIX Conf. File Storage Technol. (FAST)*, 2025.

[11] J. Kanichai, T. De Matteis, and A. Trivedi, "Building py-kvcache: A performance characterization of external KV caching for vLLM with NVMe SSDs," arXiv:2609.11744, Sep. 2026.

- [12] C. Shi, Y. Chen, Y. Chen, X. Miao, and B. Cui, "Multi-segment attention: Enabling efficient KV-cache management for faster large language model serving," arXiv:2606.02964, Jun. 2026.
- [13] Y. Liu, M. Yu, and J. Yang, "When fancy eviction fails: Rethinking cache replacement for LLM prefix reuse," arXiv:2609.28870, Sep. 2026.
- [14] S. Gao, Y. Chen, and J. Shu, "Fast state restoration in LLM serving with HCache," in *Proc. 20th Eur. Conf. Comput. Syst. (EuroSys)*, 2025.
- [15] C. Jiang, L. Gao, H. E. Zarch, and M. Annamaram, "KVPR: Efficient LLM inference with I/O-aware KV cache partial recomputation," in *Findings Assoc. Comput. Linguistics (ACL)*, 2025.
- [16] Z. Song, D. S. Berger, K. Li, and W. Lloyd, "Learning relaxed Belady for content distribution network caching," in *Proc. 17th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2020.
- [17] P. Chen *et al.*, "Toward robust and efficient ML-based GPU caching for modern inference," arXiv:2509.20979, 2025.
- [18] S. Fang *et al.*, "Not all tokens are worth caching: Learning semantic-aware eviction for LLM prefix caches," arXiv:2605.18825, 2026.
- [19] R. Pan *et al.*, "Marconi: Prefix caching for the era of hybrid LLMs," in *Proc. Mach. Learn. Syst. (MLSys)*, 2025.
- [20] D. Yang, A. Li, K. Li, and W. Lloyd, "Learned prefix caching for efficient LLM inference," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2025.
- [21] L. Moschella, L. Manduchi, and O. Sener, "Learning to evict from key-value cache," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026.
- [22] Z. Dong *et al.*, "ForesightKV: Optimizing KV cache eviction for reasoning models by learning long-term contribution," in *Proc. 43rd Int. Conf. Mach. Learn. (ICML)*, 2026.
- [23] J. Ahn *et al.*, "LookaheadKV: Fast and accurate KV cache eviction by glimpsing into the future without generation," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [24] N. Bui *et al.*, "Cache what lasts: Token retention for memory-bounded KV cache in LLMs," arXiv:2512.03324, 2025.
- [25] Z. Song *et al.*, "HALP: Heuristic aided learned preference eviction policy for YouTube content delivery network," in *Proc. 20th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2023.
- [26] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2001, pp. 973–978.
- [27] T. Lykouris and S. Vassilvitskii, "Competitive caching with machine learned advice," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 3296–3305.
- [28] N. Bansal, C. Coester, R. Kumar, M. Purohit, and E. Vee, "Learning-augmented weighted paging," in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2022.
- [29] J. Wang *et al.*, "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," in *Proc. USENIX Annu. Tech. Conf. (ATC)*, 2025.
- [30] UW SyFI Lab, "TraceLab," GitHub repository, 2026. [Online]. Available: <https://github.com/uw-syfi/TraceLab>
- [31] N. Agrawal and R. Mayer, "Benchmarking KV-cache optimizations across task quality and system performance for long-context serving," arXiv:2607.05399, May 2026.
- [32] Y. Li *et al.*, "SCBench: A KV cache-centric analysis of long-context methods," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.